

Terraform Modules (Reusable Terraform code)

What are Modules in Terraform?

In **simple terms**, a **Terraform module** is a **collection of Terraform files** (.tf) grouped together to perform a specific task — and they are **reusable**.

Why Use a Module?

Let's say you need to **create an EC2 instance** multiple times with similar settings (like AMI, instance type, security group, etc.).

Instead of copying and pasting the same code everywhere, you can:

 Create it once as a **module**

 Reuse it **anywhere** with different values (like region, instance type, etc.)

Types of Modules

1. Root Module

The main .tf files you run **terraform apply** from.

2. Child Module

A set of reusable code in another folder or source (like GitHub, Terraform Registry).

Benefits of Using Modules

Advantage	Description
 Reusable	Use the same module in different projects
 Consistent	Standardize infrastructure setup
 Easy to Test	Test small pieces before integrating
 Clean Code	Organize your code into logical components

Real-life Examples of Modules

- VPC module
- EC2 instance module
- ALB + Auto Scaling module
- S3 bucket with versioning module
- IAM user/policy module



Summary

- **Module = Reusable Terraform folder with resources**
- Helps avoid code duplication
- Encourages clean, modular, and testable infrastructure



Folder Structure

Real-World Example: EC2 Module

Imagine you want to launch EC2 instances in different environments (Dev, Prod). Instead of writing the same code again and again, you create a module and reuse it.

project-folder/

```
|
|
|— main.tf          # Root module (entry point)
|
|
└─ modules/        # Folder to hold all your reusable modules
    └─ ec2-instance/ # One module: EC2 instance
        └─ main.tf   # EC2 resource definition
        └─ variables.tf # Input variables for the module
        └─ outputs.tf  # Output values from the module
```

Final Structure (with explanation)

project-folder/

|

|— main.tf  Root module (calls the child module)

|

|— modules/  Folder to hold all reusable modules

|— ec2-instance/  FOLDER (child module)

|— main.tf  Terraform code to create EC2

|— variables.tf  Input variables for EC2

|— outputs.tf  Outputs (like EC2 instance ID)

✓ 1. Root module → [main.tf](#)

This is the entry point where you use or call the module

```
module "dev_instance" {  
    source      = "../modules/ec2-instance"  
    ami_id      = "ami-0abcdef1234567890"  
    instance_type = "t2.micro"  
    name        = "dev-server"  
}  
  
module "prod_instance" {  
    source      = "../modules/ec2-instance"  
    ami_id      = "ami-0abcdef1234567890"  
    instance_type = "t2.medium"  
    name        = "prod-server"  
}
```

Here:

- You call the module twice (once for Dev, once for Prod).
- You pass different values like `name`, `instance_type`.

✓ 2. Module code → [modules/ec2-instance/main.tf](#)

This is the actual resource definition. It uses variables from [variables.tf](#).

```
resource "aws_instance" "this" {  
  
    ami          = var.ami_id  
  
    instance_type = var.instance_type  
  
    tags = {  
  
        Name = var.name  
  
    }  
  
}
```

Here:

- It creates an EC2 instance.
- Values like `ami_id`, `instance_type` come from the root via [variables.tf](#).

✓ 3. Module variables → [modules/ec2-instance/variables.tf](#)

Defines the inputs your module expects from the root module.

```
variable "ami_id" {}  
  
variable "instance_type" {}  
  
variable "name" {}
```

✓ 4. Module outputs → [modules/ec2-instance/outputs.tf](#)

Sends information back to the root module (like EC2 instance ID).

```
output "instance_id" {  
  
    value = aws_instance.this.id  
  
}
```

In the root, you can now access:

```
output "dev_id" {  
    value = module.dev_instance.instance_id  
}
```